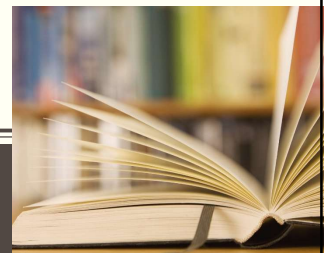
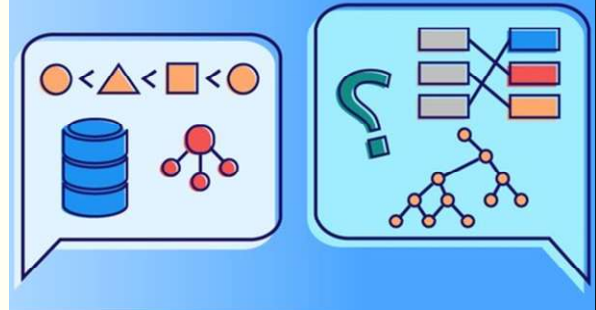


ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 07/09

- Pilas.
- Pilas implementadas en estructuras estáticas.
- Apilar – Desapilar.
- Pila implementada como TDA.
- Colas.
- Colas: Encolar y desencolar.
- Colas circulares.
- Cola implementada con TDA.

2

Contraseña del estudiante p0v14k

3

Pilas implementadas con arreglos

- Estructura de datos en la cual el acceso está limitado al elemento más recientemente insertado (el del extremo superior de la pila).
- El último elemento añadido a la pila es colocado en la cima, donde es accedido directamente, mientras que los elementos que llevan más tiempo son más difíciles de acceder.
- Operaciones naturales:

insertar - eliminar - buscar
apilar - desapilar - cima



Estructura LIFO:

“Last In, First Out”

Último en entrar, primero en salir

4

Pilas

Es una estructura de datos de entradas ordenadas

... pues hay un elemento al que se puede acceder primero (el que está en la cima de la pila), otro elemento al que se puede acceder en segundo lugar (el elemento que está justo debajo de la cima), un tercero, etc.

- Los elementos sólo se pueden agregar y eliminar en el tope.
- **Apilar:**
 - Agrega un elemento nuevo en el tope de la pila.
- **Desapilar:**
 - Elimina un elemento del tope de la pila.
 - Los elementos de la pila deben ser eliminados en el orden inverso al que se situaron en la misma.

5

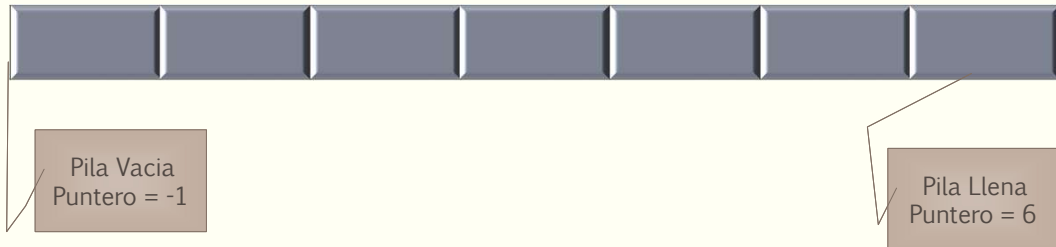
Pilas implementadas en estructuras estáticas

- Las pilas se pueden implementar guardando los elementos en un array de longitud fija. Y utilizando una variable para mantener la posición del último elemento colocado en la pila.
- Una pila puede estar vacía, o llena (en la representación con un array, si se ha llegado al último elemento).
- Si un programa intenta sacar un elemento de una pila vacía, se producirá un error por desbordamiento negativo (underflow).
- Si un programa intenta poner un elemento en una pila llena se producirá un error por desbordamiento (overflow) o rebosamiento.
- Para evitar estos errores, se diseñan funciones que comprueban si la pila está llena o vacía.

6

Pilas

Representación grafica de una pila de 7 elementos



- A medida que se agregan elementos, el puntero a la cima se modifica.
- Puntero de la pila = índice del arreglo.

7

Apilar – Insertar Elementos

- Algoritmo:

1. Verificar que la pila no este llena.
2. Incrementar en 1 el apuntador a la cima.
3. Almacenar el elemento en la posición del apuntador.

Pila vacía
ptcima = -1



Insertar 50
ptcima = 0



Insertar 25
ptcima = 1



Insertar 43
ptcima = 2

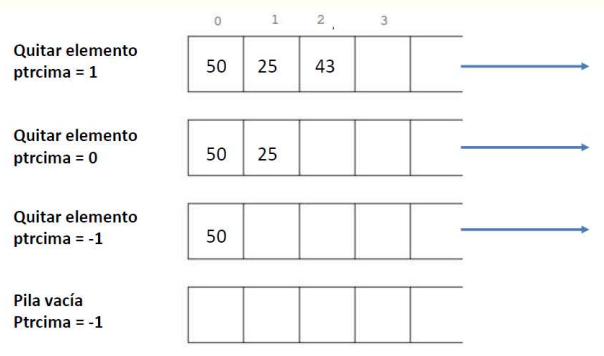


8

Desapilar – Eliminar Elementos de la pila

Algoritmo:

1. Verificar que la pila no este vacía.
2. Leer el elemento de la posición de la cima.
3. Decrementar en 1 el apuntador a la cima.



9

Ejercicio – Pila con un TDA

10

Ejercicio – Pila con un TDA

```
#include "pila.h"

int main() {
    Pila P;
    int aux;

    inicializar(P);
    mostrarPila(P);

    if (agregar(P, 3)) cout << "Se inserto 3 " << endl << endl;
    if (agregar(P, 8)) cout << "Se inserto 8 " << endl << endl;
    if (agregar(P, 5)) cout << "Se inserto 5 " << endl << endl;

    mostrarPila(P);
    cout << endl << "Invertir Pila " << endl;
    invertirPila(P);
    mostrarPila(P);

    if (sacar(P, aux)) cout << "Se retiro elemento de la pila = " << aux << endl << endl;
    else cout << "Underflow = Pila vacia " << endl << endl;
    mostrarPila(P);

    if (sacar(P, aux)) cout << "Se retiro elemento de la pila = " << aux << endl << endl;
    else cout << "Underflow = Pila vacia " << endl << endl;
    mostrarPila(P);

    if (sacar(P, aux)) cout << "Se retiro elemento de la pila = " << aux << endl << endl;
    else cout << "Underflow = Pila vacia " << endl << endl;
    mostrarPila(P);

    if (sacar(P, aux)) cout << "Se retiro elemento de la pila = " << aux << endl << endl;
    else cout << "Underflow = Pila vacia " << endl << endl;
    mostrarPila(P);
    return 0;
}
```

11

Ejercicio – Pila con un TDA

```
using namespace std;

#include "pila.h"

void inicializar(Pila & P){
    P.tope=-1;
}

bool agregar( Pila & P, int valor ){
    if (P.tope == TF-1)
        return false;    //no se puede insertar en una pila llena
    else
    {
        P.tope++;
        P.info[P.tope]= valor;
        return true;
    }
}

bool sacar ( Pila & P, int & valor ){
    if (vacía( P ))
        return false;    //no se puede sacar de una pila vacía
    else {
        valor= P.info[P.tope];
        P.tope--;
        return true;
    }
}

bool vacía ( const Pila P ) {
    return (P.tope== -1);
}
```

12

Ejercicio – Pila con un TDA

```

void mostrarPila ( const Pila P ){
    cout << "Pila = [ ";
    for (int i=0; i<=P.tope; i++)
        cout << P.info[i] << " ";
    cout << " ] , Tope = " << P.tope << endl;
}

void invertirPila ( Pila & P ){
    Pila Aux;
    int valor;

    inicializar(Aux);

    while ( not vacia(P) ){
        sacar(P, valor);
        agregar(Aux, valor);
    }

    P=Aux;
}

```

```

Pila = [ ] , Tope = -1
Se inserto 3
Se inserto 8
Se inserto 5
Pila = [ 3 8 5 ] , Tope = 2
Invertir Pila
Pila = [ 5 8 3 ] , Tope = 2
Se retiro elemento de la pila = 3
Pila = [ 5 8 ] , Tope = 1
Se retiro elemento de la pila = 8
Pila = [ 5 ] , Tope = 0
Se retiro elemento de la pila = 5
Pila = [ ] , Tope = -1
Underflow = Pila vacia
Pila = [ ] , Tope = -1

```

13

Aplicaciones de Pilas

Las aplicaciones de las pilas en la programación son numerosas. Se utilizan en:

- compiladores
- sistemas operativos
- programas de aplicaciones

14

Colas



15

Colas

- Estructura de datos en la cual se puede eliminar el elemento más antiguo (primero en entrar) y solo se puede insertar después del elemento más reciente (último en entrar).
- Se puede identificar un frente y un final.
- Operaciones naturales:
 - insertar (al final) - encolar
 - eliminar (al frente) - desencolar

Colas

- Estructura FIFO:

“First In, First Out”

Primero en entrar, primero en salir

- Ejemplo: trabajos que se mandan a impresión.

16

Colas

- Similar a una cola de pago en un supermercado.
- Es una estructura de datos de entradas ordenadas
pues solo se pueden añadir elementos por un extremo (final de la cola) y eliminar por el otro extremo (frente).

Encolar:

- Inserta un elemento nuevo al final de la cola.

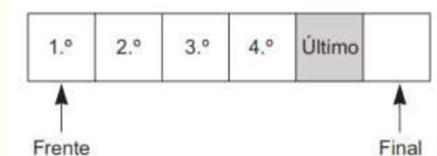
Desencolar:

- Elimina un elemento del frente de la cola..

17

Colas – Implementación en estructuras estáticas

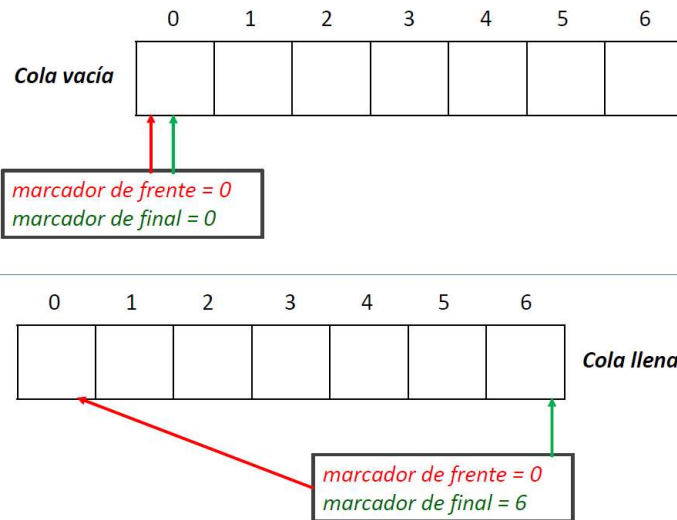
- Las colas se pueden implementar guardando los elementos en un array de longitud fija. Y utilizando dos marcadores para mantener las posiciones frente y final de la cola.
- Un marcador almacena la posición de la cabeza (frente) de la cola.
- El otro marcador almacena el primer espacio vacío que sigue al elemento final de la cola.
- Al igual que la pila, una cola puede estar vacía (sin elementos), o llena.



18

Colas – Implementación en estructuras estáticas

Representación gráfica de una cola de 6 elementos



19

Colas – Inserción de un elemento: Encolar

Cola vacía
frente = 0
final = 0



Insertar 50
frente = 0
final = 1



Insertar 25
frente = 0
final = 2

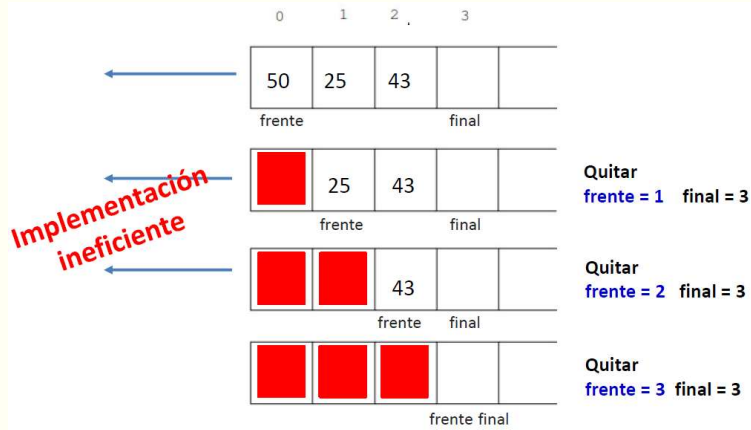


Insertar 43
frente = 0
final = 3



20

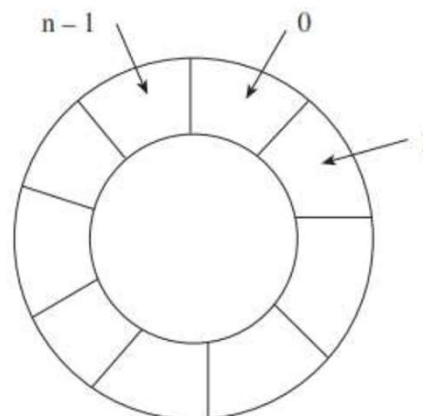
Colas – Eliminar elementos: Desencolar



21

Colas circulares en Array

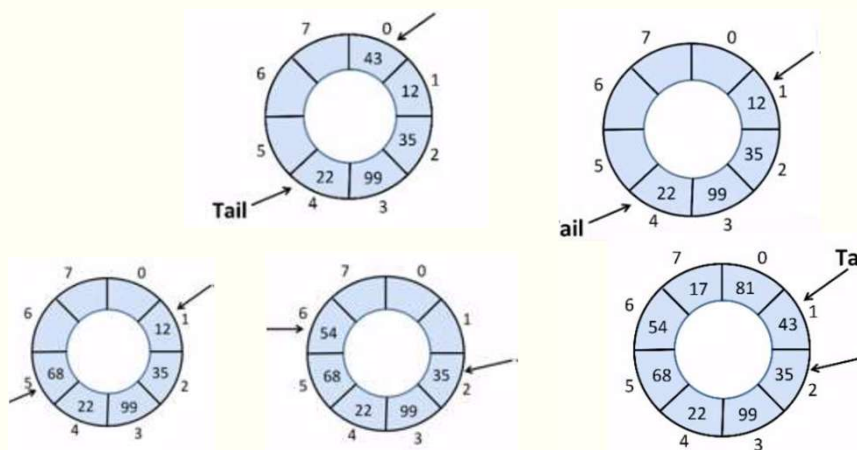
- Es la forma más eficiente de almacenar una cola en un array.
- En las colas circulares se une el extremo final con el extremo de la cabeza.
- De esta manera, la totalidad de las posiciones del array se utilizan para poder almacenar elementos de la cola sin necesidad de desplazar elementos.



22

Colas circulares con array

- Las colas circulares se implementan en un array lineal.
- Se utilizan dos marcadores para mantener las posiciones frente y final de la cola.



23

Colas circulares con Arreglos

Implementación de los índices

- final comenzó en 5, luego se puso en 6, luego en 7 Es decir, $\text{final} = \text{final} + 1$
- Pero, como la cola es circular, entonces final varía entre... 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, etc...
- Podemos usar la teoría de los restos para generar índices entre 0 y el máximo tamaño de la cola - 1:

$$\text{final} = (\text{final} + 1) \% \text{MAXTAMQ}$$

$$\text{frente} = (\text{frente} + 1) \% \text{MAXTAMQ}$$

MAXTAMQ indica el tamaño físico del arreglo = 8

- Va a quedar:

$$\text{Cuando } \text{final}/\text{frente}=4 \Rightarrow 5 \% \text{MAXTAMQ} = 5$$

$$\text{Cuando } \text{final}/\text{frente}=5 \Rightarrow 6 \% \text{MAXTAMQ} = 6$$

$$\text{Cuando } \text{final}/\text{frente}=6 \Rightarrow 7 \% \text{MAXTAMQ} = 7$$

$$\text{Cuando } \text{final}/\text{frente}=7 \Rightarrow 8 \% \text{MAXTAMQ} = 0$$

24

Colas circulares con Arreglos – Ejemplo con TDA

```

1  /* Cola de enteros implementada sobre un vector */
2  /* TDA básico y otros algoritmos relacionados con colas */
3  /* Interfase */
4  const int TF=5;
5
6  // Definicion de la estructura para la pila
7  struct Cola {
8      int info[TF]; // vector para almacenar los valores
9      int Primero, Ultimo; // subindices de frente y final
10 };
11
12 // Prototipos de Funciones
13 void inicializar(Cola & P);
14 bool encolar( Cola & P, int valor );
15 bool sacar ( Cola & P, int & valor );
16 bool vacia ( const Cola P );
17 void mostrarCola ( const Cola P );
18
19

```

25

Colas circulares con Arreglos – Ejemplo con TDA

```

#include <iostream>
#include <cstdlib>
using namespace std;

#include "cola.h"

int main(){
    Cola C;
    int aux;

    inicializar(C);
    mostrarCola(C);
    if (encolar(C, 3)) cout << "Se inserto 3 " << endl << endl;
    else cout << "Overflow" << endl;
    mostrarCola(C);
    if (encolar(C, 8)) cout << "Se inserto 8 " << endl << endl;
    else cout << "Overflow" << endl;
    mostrarCola(C);
    if (encolar(C, 5)) cout << "Se inserto 5 " << endl << endl;
    else cout << "Overflow" << endl;
    mostrarCola(C);
    if (encolar(C, 1)) cout << "Se inserto 1 " << endl << endl;
    else cout << "Overflow" << endl;
    mostrarCola(C);
    if (encolar(C, 6)) cout << "Se inserto 6 " << endl << endl;
    else cout << "Overflow" << endl;
    mostrarCola(C);
    if (sacar(C, aux)) cout << "Se retiro de la cola el elemento = " << aux << endl << endl;
    else cout << "Underflow = Cola vacia " << endl << endl;
    mostrarCola(C);
    return 0;
}

```

26

Colas circulares con Arreglos – Ejemplo con TDA

```
#include <iostream>
using namespace std;

#include "cola.h"

void inicializar(Cola & C){
    C.Primeros = C.Ultimo = 0; // P.Ultimo indica el lugar que ocupará
}

bool encolar(Cola & C, int valor){
    if ( (C.Ultimo+1)%TF == C.Primeros)
        return false;
    else {
        C.info[C.Ultimo]= valor;
        C.Ultimo= (C.Ultimo+1)%TF;
        return true;
    }
}
```

27

Colas circulares con Arreglos – Ejemplo con TDA

```
bool sacar (Cola & C, int & valor){
    if (vacía(C))
        return false; //no se puede sacar de una cola vacía
    else {
        valor= C.info[C.Primeros];
        C.Primeros = (C.Primeros+1) % TF;
        return true;
    }
}

bool vacía (const Cola C) {
    return (C.Primeros==C.Ultimo);
}

void mostrarCola ( const Cola C ){
    cout << "Cola = [ ";
    for (int i=C.Primeros; i<C.Ultimo; i= (i+1)%TF )
        cout << C.info[i] << " ";
    cout << " ] , Primeros = " << C.Primeros << " , Ultimo = " << C.Ultimo << endl;
}
```

```
Cola = [ ], Primeros = 0 , Ultimo = 0
Se inserto 3
Cola = [ 3 ], Primeros = 0 , Ultimo = 1
Se inserto 8
Se inserto 5
Se inserto 1
Overflow
Cola = [ 3 8 5 1 ], Primeros = 0 , Ultimo = 4
Se retiro de la cola el elemento = 3
Cola = [ 8 5 1 ], Primeros = 1 , Ultimo = 4
```

28

Aplicaciones de Colas

- Las aplicaciones de las colas en la programación son numerosas. Se utilizan en:
 - colas de mensajes
 - colas de tareas a realizar por una impresora
 - colas de prioridades

29

Algoritmos y Estructuras de Datos – AEDD – Asistencia 07/09

Contraseña del estudiante
p0v14k

30

Algoritmos y Estructuras de Datos – AEDD – Agenda 07/09

- ✓ Pilas.
- ✓ Pilas implementadas en estructuras estáticas.
- ✓ Apilar – Desapilar.
- ✓ Pila implementada como TDA.
- ✓ Colas.
- ✓ Colas: Encolar y desencolar.
- ✓ Colas circulares.
- ✓ Cola implementada con TDA.

¡OBJETIVO CUMPLIDO!

31

Para profundizar

Libro: Resolución de problemas con C++ - Savitch

Capítulo 10

(Pág. 545)

Capítulo 6

Libro: Benjumea-Roldan

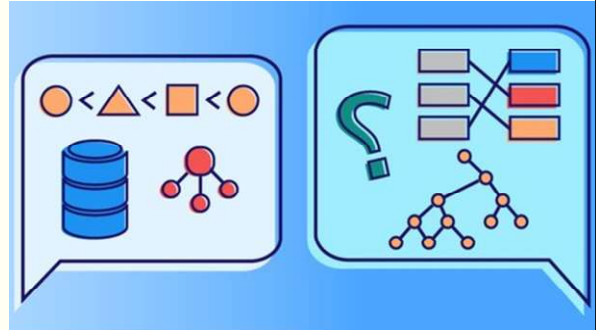
Universidad de Málaga

32

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2025 – Cursado Anual

¿DUDAS?



33

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2025 – Cursado Anual

¡Muy buen fin de semana!



34